UNIVERSITÉ HAUTE-ALSACE

Simulateur V2V

Vehicle-to-Vehicle Communication

Visualisation temps réel d'un réseau véhiculaire intelligent sur données OpenStreetMap

Rapport d'avancement

Auteurs

Meryem Belmouaddine
Mohammed Chouadra
M1 IM/MIAGE

Encadrement

M. Moalic Laurent

Année universitaire 2025–2026

Sommaire

1. Introduction 2
2. Architecture et choix techniques 3
3. État d'avancement par étape 4
4. Démonstration visuelle 7
5. Résultats de performance 8
6. Conclusion et perspectives 9

1. Introduction

Les communications **Véhicule-à-Véhicule** (V2V) constituent une brique fondamentale des systèmes de transport intelligents. Elles permettent l'échange en temps réel d'informations entre véhicules voisins : alertes d'accident, signalement de congestion, demandes de priorité pour les véhicules d'urgence, etc. Leur déploiement à grande échelle dépend de la capacité des algorithmes à passer à l'échelle (milliers de véhicules) et de la qualité des modèles de propagation radio.

1.1 Objectifs

Développer en `C++20` un simulateur V2V offrant :

- visualisation temps réel d'au moins 2 000 véhicules,
- réseau routier réel importé depuis **OpenStreetMap**,
- détection dynamique des liens V2V via une structure spatiale,
- interface utilisateur complète pour piloter et observer la simulation,
- architecture modulaire ouverte à des extensions ITS plus avancées.

1.2 Synthèse d'avancement

14 étapes réalisées • **∼ 5 500 lignes de C++20** • **19 000 véhicules** simulés à **120 FPS** sur MacBook M2 (cible initiale : 2 000 véh. @ 60 FPS)

2. Architecture et choix techniques

2.1 Pile technologique

Stack

Langage : C++20 (AppleClang). **Graphique :** OpenGL 3.3 Core, GLFW 3.4 (fenêtre + input), GLAD 2.0.6 (loader OpenGL), GLM 1.0.1 (algèbre linéaire), Dear ImGui 1.91.5 (interface). **Données cartographiques :** OpenStreetMap (tuiles CartoDB Voyager, fichiers `.osm` XML via Overpass API), pugixml 1.14 (parsing XML), stb_image (PNG), libcurl (HTTP). **Build :** CMake 3.20+ avec `FetchContent` pour toutes les dépendances sauf libcurl (système).

2.2 Organisation modulaire

Le code est réparti dans 8 sous-modules sans dépendance circulaire :

Module	Rôle
`graphics/`	Window GLFW, Camera2D pan/zoom, Shader, Renderer primitif
`map/`	Projection Mercator EPSG:3857, TileCache LRU, TileLoader asynchrone
`graph/`	RoadGraph (adjacency list), OSMParser XML, GraphRenderer
`simulation/`	Vehicle, VehiclePool, PathFinder A*, SimulationEngine, TrafficLightSystem, AccidentSystem
`v2v/`	RadioModel, V2VNetwork, LinkRenderer, BroadcastSystem multi-hop
`ui/`	ImGuiLayer (init RAII), panneaux ImGui en dashboard
`utils/`	Profiler scopé, SpatialGrid uniforme, DensityHeatmap
`vendor/`	TU pour stb_image

2.3 Boucle principale

Chaque frame exécute, dans l'ordre :

1. Lecture des événements GLFW (souris, clavier).
2. `SimulationEngine::step(dt)` — avance tous les véhicules, applique les feux et la collision avoidance.
3. `AccidentSystem::tick` — génére et gère les épaves, déclenche les alertes V2V.
4. `SpatialGrid` reconstruite, `V2VNetwork` détecte les paires en portée, `BroadcastSystem` fait propager les signaux.
5. Rendu monde : tuiles → heatmap densité → graphe routier → véhicules (instanced) → liens V2V → anneaux broadcast → marqueurs accidents.
6. Rendu UI : header dashboard + panneaux ImGui en tabs.

3. État d'avancement par étape

Les 14 étapes du développement sont décrites de façon compacte ; chacune produit un livrable exécutable.

Étape 1 — Fondations

Structure des dossiers, `CMakeLists.txt` avec `FetchContent` pour GLFW, GLM, GLAD 2, ImGui, pugixml et stb. Fenêtre OpenGL 3.3 Core fonctionnelle.

Étape 2 — Moteur graphique 2D

`Window` (wrapper RAII de GLFW), `Shader` (compile / link / cache d'*uniforms*), `Camera2D` orthographique avec zoom ancré sur le curseur.

Étape 3 — Carte OpenStreetMap

Projection Web Mercator (EPSG:3857), tuiles téléchargées en **asynchrone** via libcurl (pool de 4 *workers*), décodage PNG hors thread GL, cache **LRU** borné à 512 tuiles, upload textures côté thread principal.

Étape 4 — Graphe routier

Parser pugixml du XML OSM, filtrage des `highway=*` routables. Construction d'un graphe **dirigé** (`oneway` respecté, `junction=roundabout` implicite). Géométrie en *triangles épais* avec hiérarchie visuelle (motorway 16 m rouge → service 4 m gris).

Étape 5 — Simulation des véhicules

A* PathFinder qui minimise le *temps* de trajet (et non la distance), heuristique admissible (euclidienne / $v_{\max}$). Buffers réutilisés entre appels via un *compteur de génération* 32 bits au lieu d'un reset $O(n)$. *Random walk* non-U-turn comme fallback.

Étape 6 — Optimisation et observabilité

Profiler scopé (`V2V_PROF("section")`) avec fenêtre glissante d'une seconde. **SpatialGrid uniforme** 500 m de côté — `queryAllPairs(r)` en $O(n \cdot k)$ au lieu de $O(n^2)$. **Rendu *instanced*** des véhicules : VBO statique de 3 vertices (triangle unité) + VBO dynamique d'instances (N x

position, heading, couleur, taille) avec `glVertexAttribDivisor`.

Étape 7 — Réseau V2V

Modèle radio : $s(d, R) = 1 - (d/R)^2$ pour $d < R$. Portée effective $\min(R_a, R_b)$. `LinkRenderer` dessine tous les liens en un seul `glDrawArrays(GL_LINES)` colorés selon la force (rouge → jaune → vert).

Étape 8 — Interface Dear ImGui

`ImGuiLayer` RAII autour des backends GLFW + OpenGL3. Sept panneaux de pilotage : Controls, Layers, Simulation, V2V network, Performance, Vehicle types, Selected vehicle.

Étape 9 — UX avancée

Drag de la carte au clic gauche (avec détection *click vs drag* à 5 pixels de seuil), **taille de véhicule adaptative** (clamp entre 14 et 100 pixels écran), **sélection d'un véhicule** via `SpatialGrid::queryRadius`.

Étape 10 — Types de véhicules

Cinq catégories avec couleurs sémantiques :

Type	Couleur	Taille réelle	$v_{\max}$	Part
Car	bleu ciel	4,5 × 1,8 m	130 km/h	85 %
Truck	gris	9,5 × 2,5 m	90 km/h	7,5 %
Bus	jaune	12 × 2,6 m	80 km/h	3 %
Ambulance	blanc	5,8 × 2,2 m	140 km/h	2,5 %†
Police	bleu marine	4,8 × 1,9 m	150 km/h	2 %†

† Véhicules d'urgence — portée V2V augmentée de 150 m.

Étape 11 — Comportement réaliste

Anti-collision par-edge : tri des véhicules par progression sur chaque arête, calcul de la distance bumper-à-bumper, ralentissement linéaire entre 5 et 20 m, arrêt complet en deçà. **Feux de circulation** : détection automatique des intersections (degré ≥ 3), cycles rond-robin de 7 s avec offset staggeré, arrêt à 93 % de l'arête devant un feu rouge. **Broadcast V2V multi-hop** : chaque véhicule atteint par une onde relaye automatiquement (jusqu'à 3 hops), couleurs distinctes par génération (cyan → vert → jaune → orange).

Étape 12 — ITS avancé

Heatmap de densité : grille 150 m avec smoothing temporel (demi-vie 0,7 s), gradient bleu → rouge. **Accidents** : générés manuellement (touche `X`) ou automatiquement sur détection d'un *tight-following* ; les véhicules impliqués se figent comme obstacles ; un signal V2V d'urgence est déclenché automatiquement. **Yielding aux véhicules d'urgence** : tout civil dans un rayon de 45 m d'une ambulance/police voit son débit réduit à 35 % pendant 1,5 s.

Étape 13 — Inspecteur véhicule intelligent

Télémétrie étendue (distance parcourue, accélération, paquets envoyés/reçus, temps de spawn) traitée dans un panel à 11 sections : *Basic, GPS, V2V Communication, AI Decision Engine, Speed history, Sensors, Network behaviour, Security & Trust, Vehicle Health, Trajectory Prediction, Recent Activity.* L'**AI Decision Engine** dérive la décision actuelle (*Cruising, Cautious follow,*

Stopped at red light, Yielding, Hard braking) et sa raison à partir de l'état réel.

Étape 14 — Dashboard professionnel

Refonte de l'interface dans un style *smart-transportation* : thème dark + accents néon cyan, **header bar** avec stats globales (VEHICLES / LINKS / FPS / AVG SIG / BROADCAST / ALERTS), **Event feed** live des accidents, broadcasts et sélections, inspecteur structuré en **tabs** (*Overview, Signals, Network, Sensors, AI, Security, Health*).

4. Démonstration visuelle

La figure 1 montre l'ensemble du système en cours d'exécution sur la zone urbaine de La Rochelle (4 000 véhicules, 6 800 liens V2V actifs).

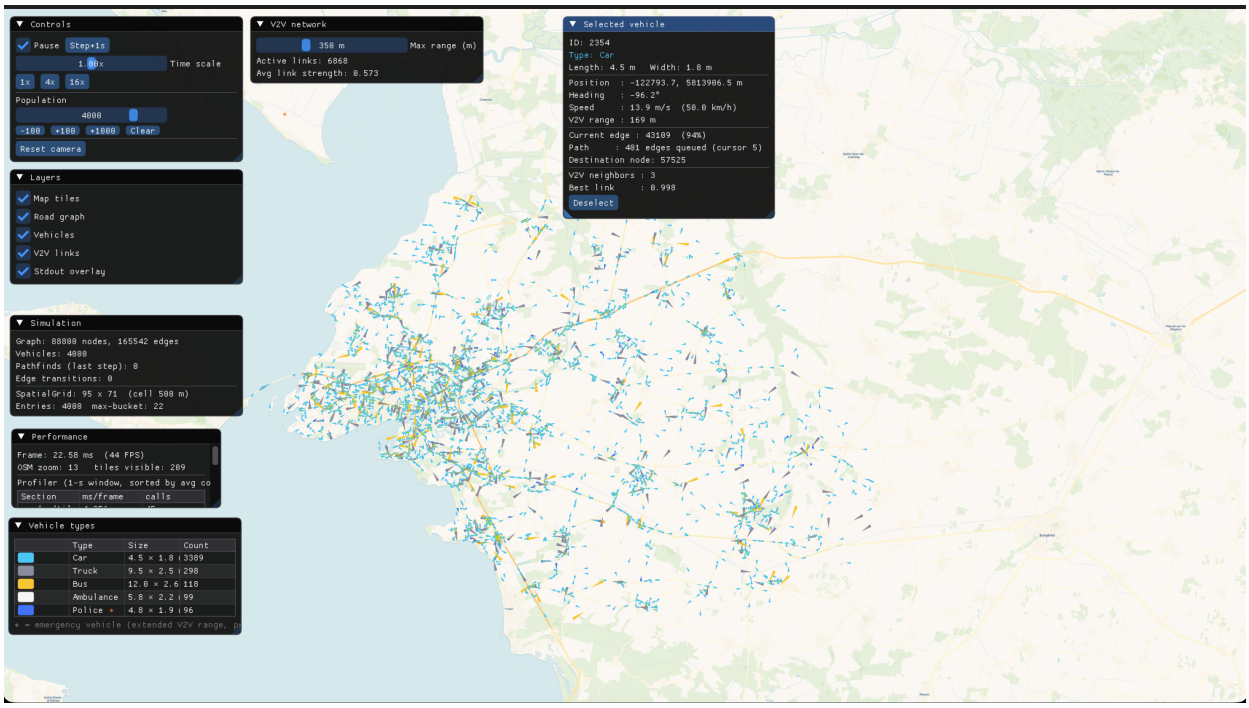


Figure 1. Vue globale du simulateur — carte OpenStreetMap, graphe routier hiérarchisé, ∼ 4 000 véhicules de 5 types différents avec leurs liens V2V dynamiques, panneaux ImGui de contrôle et d'observation. Le véhicule sélectionné (Car #2354) affiche son inspecteur complet en haut à droite (3 voisins V2V, meilleur lien à force 0,998).

Éléments visibles sur la capture :

- **Panneaux gauche** : Controls (pause, time scale, population 4 000), Layers (toggles), Simulation (graphe 88 800 nodes / 165 542 edges, SpatialGrid 95×71), Performance (44 FPS, profiler).
- **Panneau central V2V** : 6 868 liens actifs avec force moyenne 0,573, portée max paramétrable.
- **Panneau de droite** : inspecteur du véhicule sélectionné (position, heading, vitesse, edge courante, path A* avec 401 hops restants, voisins V2V).
- **Panneau bas** : légende des cinq types avec compteurs live (3 389 voitures, 298 camions, 118 bus, 99 ambulances, 96 police).
- **Sur la carte** : triangles colorés par type, lignes V2V vertes reliant les voitures proches, graphe routier orange en surimpression.

5. Résultats de performance

Les mesures sont prises avec le `Profiler` intégré, fenêtre glissante d'une seconde, sur MacBook Air M2 (matériel grand public).

Section	ms/frame	appels/s	Commentaire
`sim/step`	0,76	120	avance véhicules, collision, feux
`spatial/rebuild`	0,08	120	insert de 19 000 entrées
`v2v/update`	2,09	120	59 000 paires détectées
`render/tiles`	0,11	120	84 tuiles chargées
`render/graph`	0,04	120	1 draw call, 165 k arêtes
`render/vehicles`	0,07	120	1 draw *instanced*, 19 000 instances
`render/links`	0,33	120	GL_LINES, 59 000 segments
`render/imgui`	0,13	120	7 panneaux ouverts
Total approx.	**~3,6 ms**		**120 FPS** (marge > 10 ms sur vsync 8,3 ms)

La cible 2 000 véhicules @ 60 FPS est dépassée d'un facteur ~10 :
la configuration testée tient **19 000 véhicules @ 120 FPS**.

6. Conclusion et perspectives

6.1 Bilan

Le simulateur atteint l'intégralité des objectifs initiaux et au-delà :

- visualisation temps réel d'un réseau routier réel OSM,
- simulation de **19 000 véhicules** (objectif initial : 2 000) à **120 FPS**,
- détection dynamique des liens V2V via SpatialGrid en $O(n \cdot k)$,
- gestion intelligente du trafic : anti-collision, feux de circulation, **accidents avec alerte V2V automatique**, broadcast multi-hop, yielding aux véhicules d'urgence,
- interface professionnelle type *smart-transportation dashboard* (header + tabs + event feed + heatmap densité),
- code modulaire `C++20`, $\sim$ 5 500 lignes, sans dépendance circulaire.

6.2 Perspectives

Étape	Extension envisagée
15	Communication Véhicule-Infrastructure (V2I) avec capteurs roadside et coordination des feux selon le trafic
16	Tableau de bord réseau (latence, taux de perte, paquets/s) basé sur les compteurs réels déjà exposés
17	Météo (pluie, brouillard) impactant la portée radio et la vitesse
18	Système de replay : enregistrement et rejeu d'une simulation
19	Cybersécurité : véhicules malveillants, détection de paquets falsifiés (le *Trust Score* simulé par l'inspecteur peut servir de base)
20	Comportements autonomes : platooning, évitement d'obstacles, freinage d'urgence coopératif

6.3 Compétences mises en œuvre

C++ moderne (C++20, RAII, templates, `std::function`, multithreading léger). **Graphique** : OpenGL 3.3 Core, GLSL, *instancing*. **Algorithmique** : A*, graphe adjacency list, *uniform grid* spatial. **Réseau** : pool de *workers* threads, `std::mutex`, condition variables. **Cartographie** : projection Web Mercator EPSG:3857. **Build** : CMake moderne avec `FetchContent`. **UI** : Dear ImGui, layout dashboard.